SENG 31242 - System Design Project (24/25)

Project – CivilCore

Group - Data Dynamos

- SE/2022/001
- SE/2022/006
- SE/2022/018
- SE/2022/036
- SE/2022/045

Software Design Specification Draft

---

1. Introduction

1.1 Purpose

The primary purpose of this Software Design Specification (SDS) is to establish the complete architectural and detailed system design for the **CivilCore** application. CivilCore is a simplified construction site management application designed specifically for civil engineering teams to replace overly complex tools like Primavera P6. This document translates the business needs and user constraints defined in the Software Requirements Specification (SRS) into a concrete technical blueprint. It will serve as the core reference document for the development team during the upcoming SENG 34213 development phase, providing exact specifications for system components, database schemas, and interface boundaries.

1.2 Scope

The scope of this design covers the entire CivilCore software ecosystem. The system is designed for three distinct user roles: Owners, Managers, and Supervisors. To serve these users, the design details the architecture of the following components:

- A **cross-platform mobile application** built using **Flutter** specifically for on-site Supervisors.
- A **web-based dashboard** built using **React** for Managers and project Owners.
- A central **RESTful backend API** built with **Node.js** to handle system logic.

- A **relational database (MySQL)** for strict data integrity, supplemented by **cloud object storage (S3)** to handle heavy geo-tagged site photos and project documents.

The system design encompasses core functionalities including daily reporting, Bill of Quantities (BOQ) tracking, asset requests, issue logging, and robust offline data synchronization. It explicitly excludes the design of complex resource levelling, vendor procurement, or financial accounting modules, as these are out of scope for the CivilCore product.

1.3 Definitions, Acronyms, and Abbreviations

- **API:** Application Programming Interface, enabling communication between the frontend applications and the backend.
- **BOQ:** Bill of Quantities, representing the imported list of work items and their planned quantities that supervisors will report against.
- **JWT:** JSON Web Token, utilized for stateless user authentication and role-based access control.
- **P6:** Primavera P6, the complex enterprise software that CivilCore is designed to replace.
- **RBAC:** Role-Based Access Control, ensuring users can only access features permitted for their assigned role (Owner, Manager, Supervisor).
- **SDS:** Software Design Specification.
- **SRS:** Software Requirements Specification.

1.4 Overview of Document

- **Section 2** provides a system overview and relationship to the SRS.
- **Section 3** details the high-level Architectural Design, including the Component Diagram, Deployment Diagram, and formal Architectural Decision Records (ADRs) justifying the technology stack.
- **Section 4** opens the "black box" of the system with Detailed Design elements, including the Class Diagram, Database Data Model, and Sequence Diagrams mapping the internal logic for all use cases.
- **Section 5** outlines the UI Design through wireframes, navigation flow diagrams, and UX decisions.
- **Sections 6, 7, and 8** finalize the design by specifying internal/external interfaces, security implementations (including OWASP mitigations), and how the system satisfies Non-Functional Requirements.

2. System Overview

2.1 System Summary

CivilCore is a streamlined, mobile-first construction site management platform tailored specifically for civil engineering teams. It is strategically designed to replace heavy enterprise software like Primavera P6, which field teams often find too cumbersome for daily site operations. The system focuses strictly on core field tracking and management capabilities, deliberately excluding complex resource levelling, financial accounting, and vendor procurement modules to ensure a lightweight user experience.

The system serves three primary user roles across two distinct application platforms:

- **Supervisors (Mobile App):** Operating on a cross-platform Flutter mobile application, field supervisors can submit daily progress reports directly from the construction site. The application enables them to log completed tasks directly linked to the project's Bill of Quantities (BOQ), track labour hours (skilled and unskilled), record materials utilized, request future assets, and log on-site issues utilizing geo-tagged photos. Crucially, this application features robust offline capabilities, allowing supervisors to fill out reports without an active internet connection and automatically syncing the data when connectivity is restored.
- **Managers (Web Desktop App):** Utilizing a React-based web dashboard, managers are equipped to set up new projects, import BOQs (via CSV/Excel), assign supervisory teams, and review/approve incoming asset and material requests. The dashboard provides comprehensive progress monitoring, tracking actual project completion percentages against BOQ quantities, as well as monitoring materials used versus the allocated budget.
- **Owners (Web Desktop App):** Owners access a read-only global portfolio dashboard that provides a high-level summary across all active projects. This interface highlights overall progress, cost-versus-budget metrics, and surface-level system alerts regarding schedule delays or budget overruns.

2.2 Relationship to Software Requirements Specification (SRS)

This Software Design Specification is directly derived from the approved CivilCore Software Requirements Specification (SRS). The architectural decisions, database schemas, and interface designs outlined in the subsequent chapters provide the concrete technical blueprint required to realize both the functional and non-functional requirements established during the analysis phase.

Specifically, this design physically models and accommodates the 10 core use cases defined in the SRS, ranging from daily reporting and progress monitoring to subcontractor tracking and document library management. Furthermore, the system architecture specifically resolves the critical constraints and success criteria identified in the SRS:

- **Performance & Usability Constraint:** The mobile user interface and backend database query structures are optimized to ensure a supervisor can successfully complete and submit a daily report in under 5 minutes.
- **Reliability Constraint:** The implementation of local caching within the mobile application fulfills the strict requirement for offline reporting capabilities in remote site locations with poor network coverage.

3. Architectural Design

3.1 Architecture Pattern

The CivilCore system implements a decoupled **Client-Server Architecture** operating via a stateless RESTful backend. This pattern was selected over a monolithic architecture to enforce a strict separation of concerns between the user interfaces and the core business logic.

Justification referencing Quality Attributes:

- **Scalability:** By decoupling the client applications from the server, the central Node.js REST API can be scaled horizontally and independently to handle high volumes of concurrent requests, such as the "end-of-day rush" when multiple supervisors submit heavy daily reports simultaneously.
- **Maintainability:** The frontend applications (Flutter for mobile, React for web) and the backend API exist in separate repositories and deployment pipelines. This allows the web and mobile development teams to work in parallel without causing merge conflicts or deployment bottlenecks, directly supporting the tight 4-month development timeline.
- **Reliability:** A stateless REST API architecture simplifies the implementation of the strict offline-mode requirements. Since the backend does not hold active session states for the mobile client, the Flutter app can seamlessly cache data locally in SQLite when the network drops, and safely push these complete JSON payloads to the API once connectivity is restored.

3.2 Technology Stack & Justifications

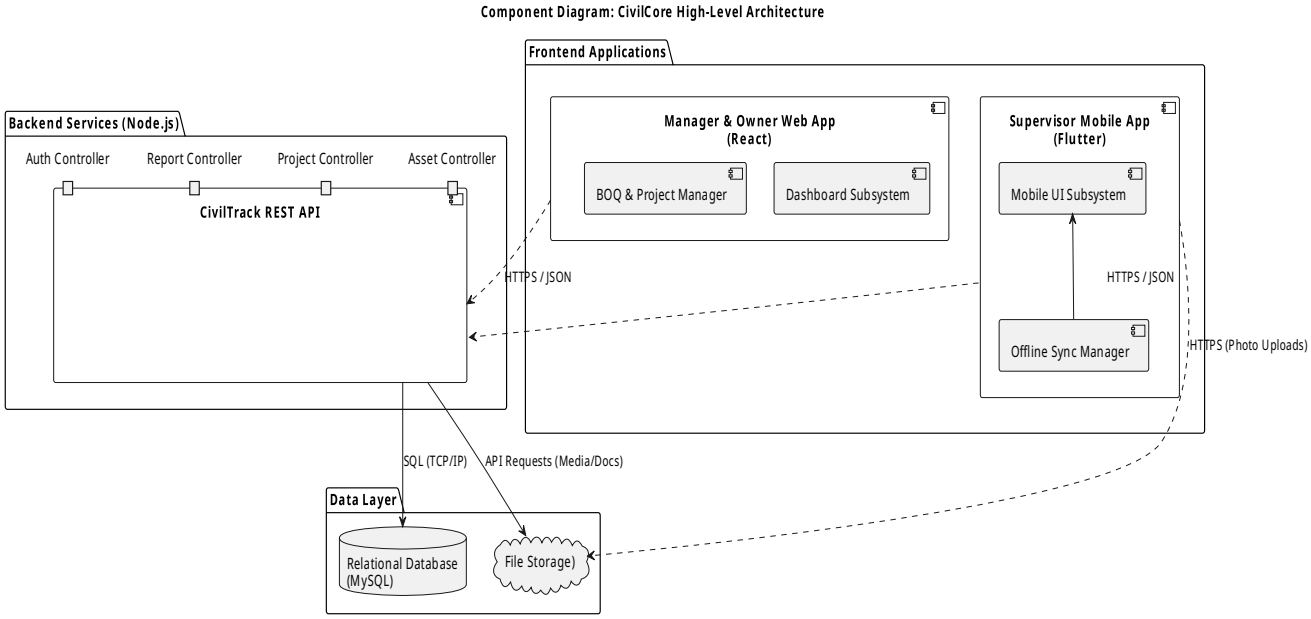
Every major component selected below has been formally evaluated against alternatives. The detailed rationales, alternatives considered, and accepted trade-offs for these decisions are fully documented in the project's **Architectural Decision Records (ADRs)** located in the documents/architecture/decisions/ repository.

- **Mobile Application: Flutter (Dart)**
  - *Decision:* Adopted to build the field supervisor mobile application for both iOS and Android.
  - *Justification:* Flutter allows for a single, unified cross-platform codebase, drastically reducing the maintenance overhead. Its custom rendering engine guarantees pixel-perfect UI consistency across various mobile devices, ensuring the large touch targets required for site supervisors function reliably.
  - *Reference:* ADR-01-Flutter.md
- **Web Application: React (JavaScript)**
  - *Decision:* Adopted to build the web-based dashboards for Managers and Owners.

    - *Justification:* The Manager and Owner dashboards require complex, data-heavy visualizations (such as Gantt-style schedules and dynamic budget tracking). React's Virtual DOM provides highly efficient rendering of these large datasets without noticeable UI lag, directly addressing our performance requirements.
    - *Reference:* ADR-02-React.md
- **Backend API: Node.js (Express)**
    - *Decision:* Adopted to build the central RESTful API.
    - *Justification:* Node.js utilizes an asynchronous, non-blocking I/O model. This is exceptionally efficient for handling multiple concurrent network requests, making it the ideal framework to process the heavy, simultaneous data payloads submitted by field supervisors at the end of their shifts.
    - *Reference:* ADR-03-NodeJS.md
- **Primary Database: MySQL**
    - *Decision:* Adopted as the primary relational database management system.
    - *Justification:* While PostgreSQL was suggested in the initial client brief, the team explicitly pivoted to MySQL. The system requires strict ACID compliance to securely manage the highly relational data (Projects containing Tasks, which belong to Daily Reports). MySQL provides this necessary data integrity, and the development team possesses deep, existing expertise in the MySQL engine, significantly reducing technical risk during the development phase.
    - *Reference:* ADR-04-MySQL.md
- **File Storage: AWS S3 (Cloud Object Storage)**
    - *Decision:* Adopted for handling heavy binary files.
    - *Justification:* To keep the relational database performant and lightweight, MySQL will only store metadata and file URLs. The actual binary blobs for geo-tagged site photos, issue logs, and project documents (DWG/PDF files) will be pushed directly to AWS S3 via the backend API.

3.3 Component Diagram

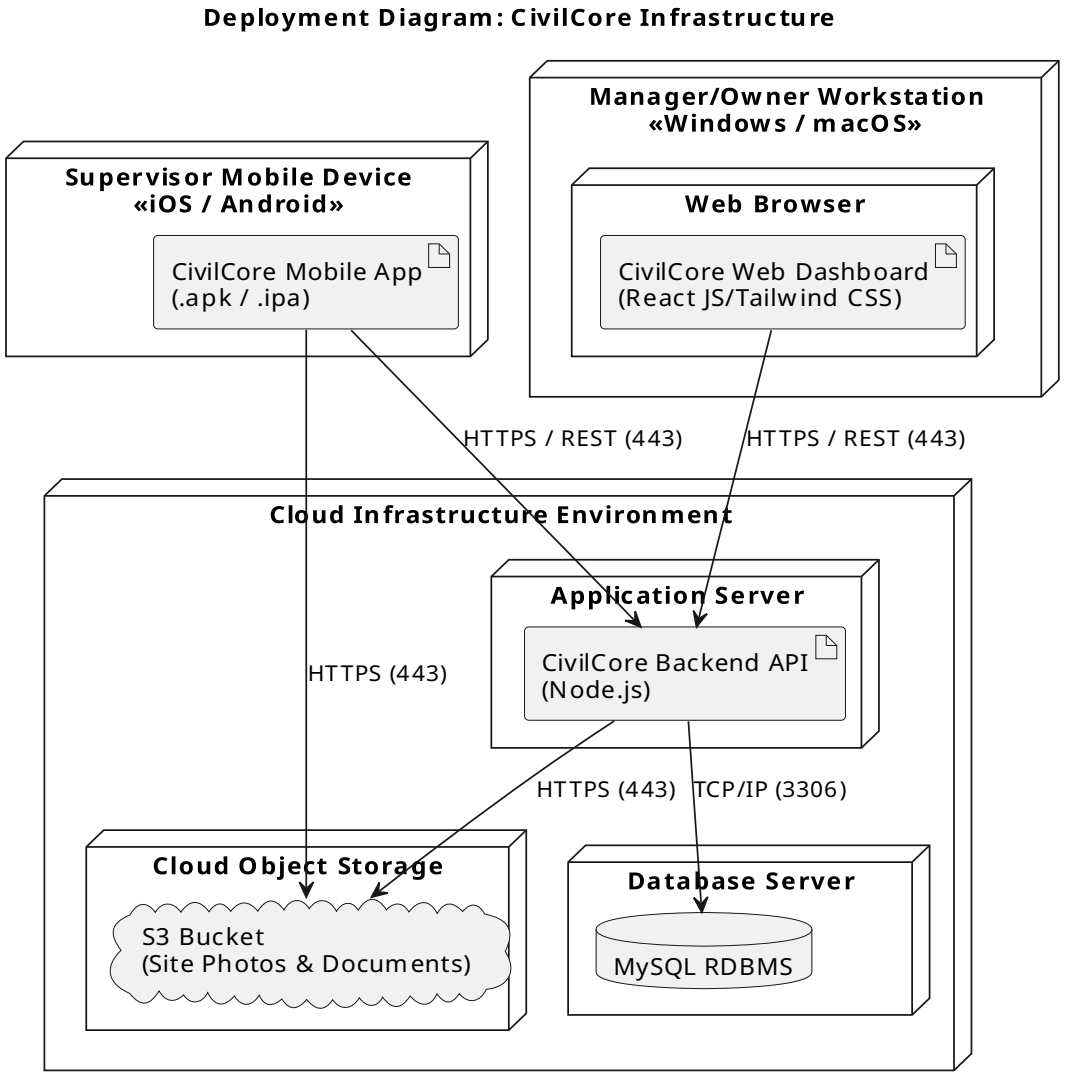
The following Component Diagram illustrates the high-level logical architecture of the CivilCore system. It maps the distinct software packages (Mobile App, Web App, REST API, Database, and S3 Storage) and traces the JSON-over-HTTPS interfaces connecting them.



(Note: The source PlantUML file for this diagram is located at diagrams/component-diagram/)

3.4 Deployment Diagram

The Deployment Diagram models the physical infrastructure topology of the CivilCore system. It demonstrates where each software artefact executes in the production environment, tracking the flow from the Supervisor's physical iOS/Android device in the field to the cloud-hosted Application and Database servers.



(Note: The source PlantUML file for this diagram is located at diagrams/deployment-diagram/)

4. Detailed Design

4.1 Database and Data Model Design

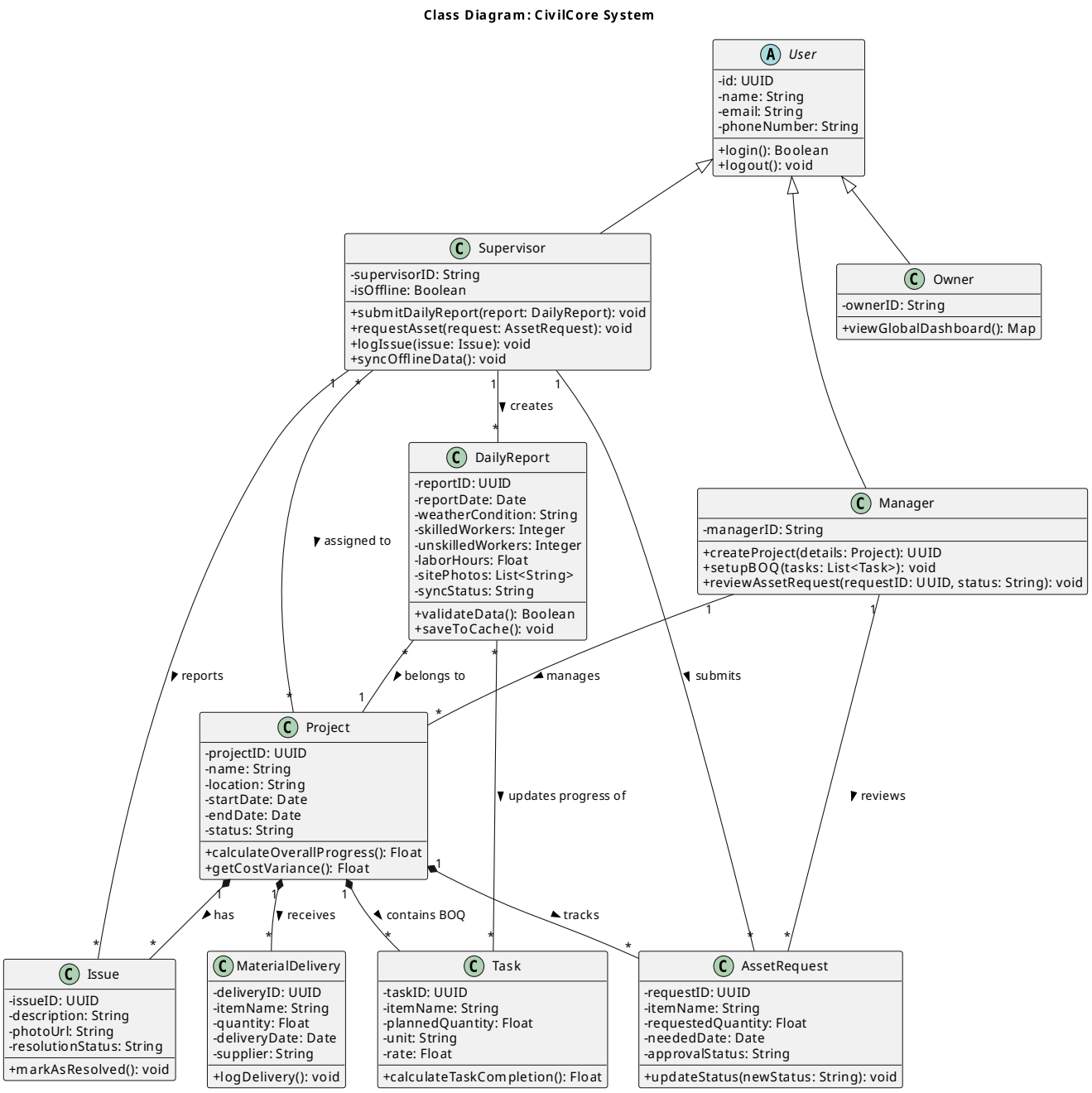
The CivilCore system utilizes a highly relational data model implemented in MySQL to ensure strict ACID compliance across all field reporting and management activities. The core entity of the system is the Project.

The primary entities and their relationships are structured as follows:

- **Project:** Stores overarching details including the project name, location, start and end dates, and the assigned team of supervisors. It acts as the root node for all other relational data.
- **Task:** Represents individual line items imported from the Bill of Quantities (BOQ). Each task is strictly linked to a single Project and contains the planned quantity, unit of measurement, and assigned rate.
- **Daily Report:** Represents a single end-of-day submission by a field supervisor. Each report is relationally mapped to the submitting Supervisor and the target Project. It contains the report date, weather conditions, logged labour hours (categorized by skilled and unskilled workers), and an array of geo-tagged site photo URLs.
- **TaskProgress (Join Table):** Maps the many-to-many relationship between a Daily Report and a Task. It records the exact quantity of work completed for a specific task on a specific day.
- **Asset Request:** Tracks requests for equipment or materials needed for upcoming work. It includes the requested item, quantity, needed date, and current approval status, linked to a specific Project and requesting Supervisor.
- **Issue:** Represents a snag or problem logged on-site. It contains a description, a photo URL (stored in S3), the reporting supervisor, and the resolution status.
- **Material Delivery:** An independent log tracking materials arriving on site, which is distinct from the materials actively used in daily reports. It records the item, quantity, delivery date, and supplier.

4.2 Class Diagram

The system class diagram formally defines the object-oriented structure of the backend application, outlining the attributes, methods, and visibility of all system entities. It models the core inheritance strategy where Supervisor, Manager, and Owner inherit from a base User class to handle authentication, while encapsulating the distinct methods available to each role.

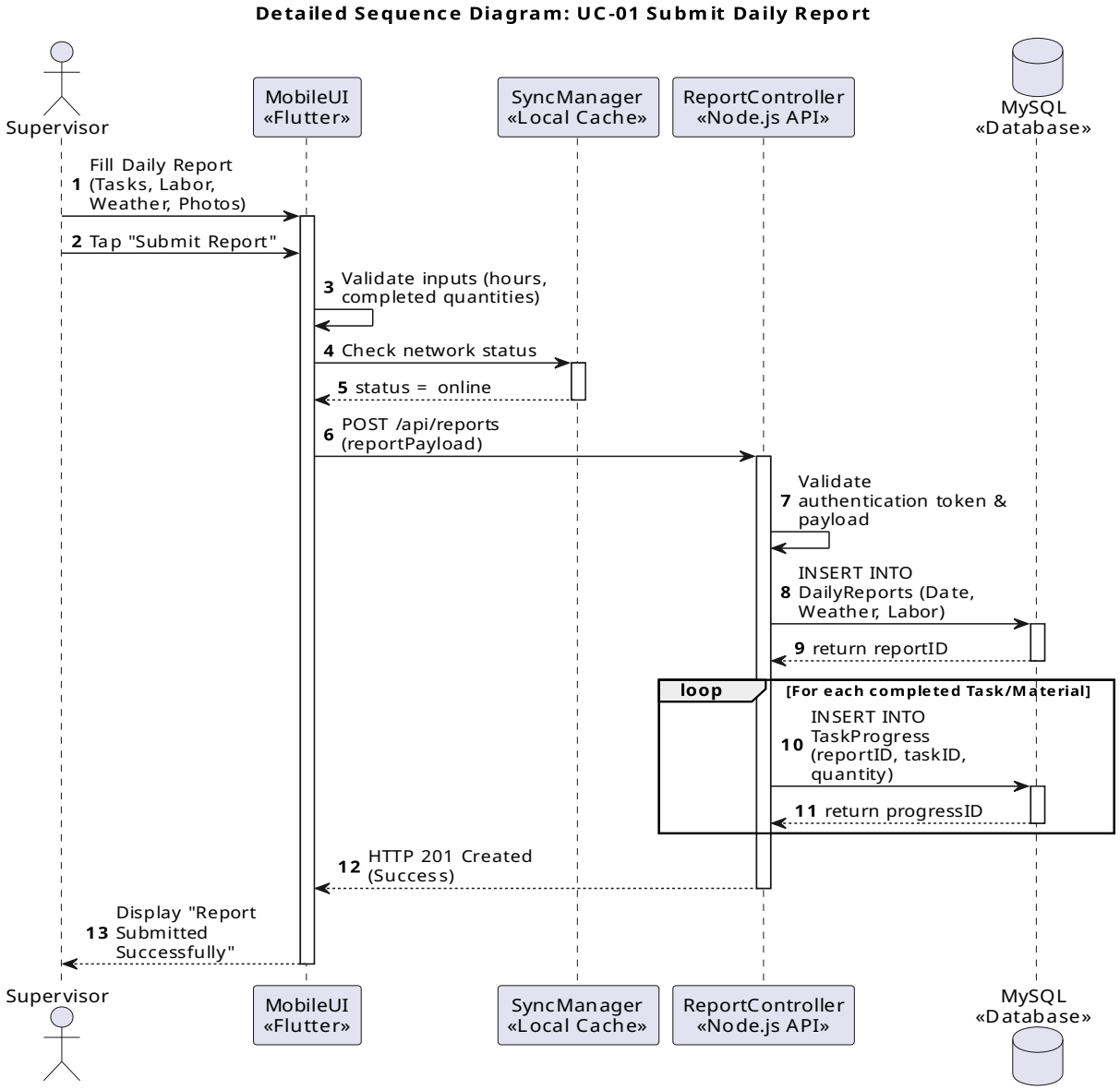


(Note: The source PlantUML file for this diagram is located at documents/diagrams/class-diagram.puml)

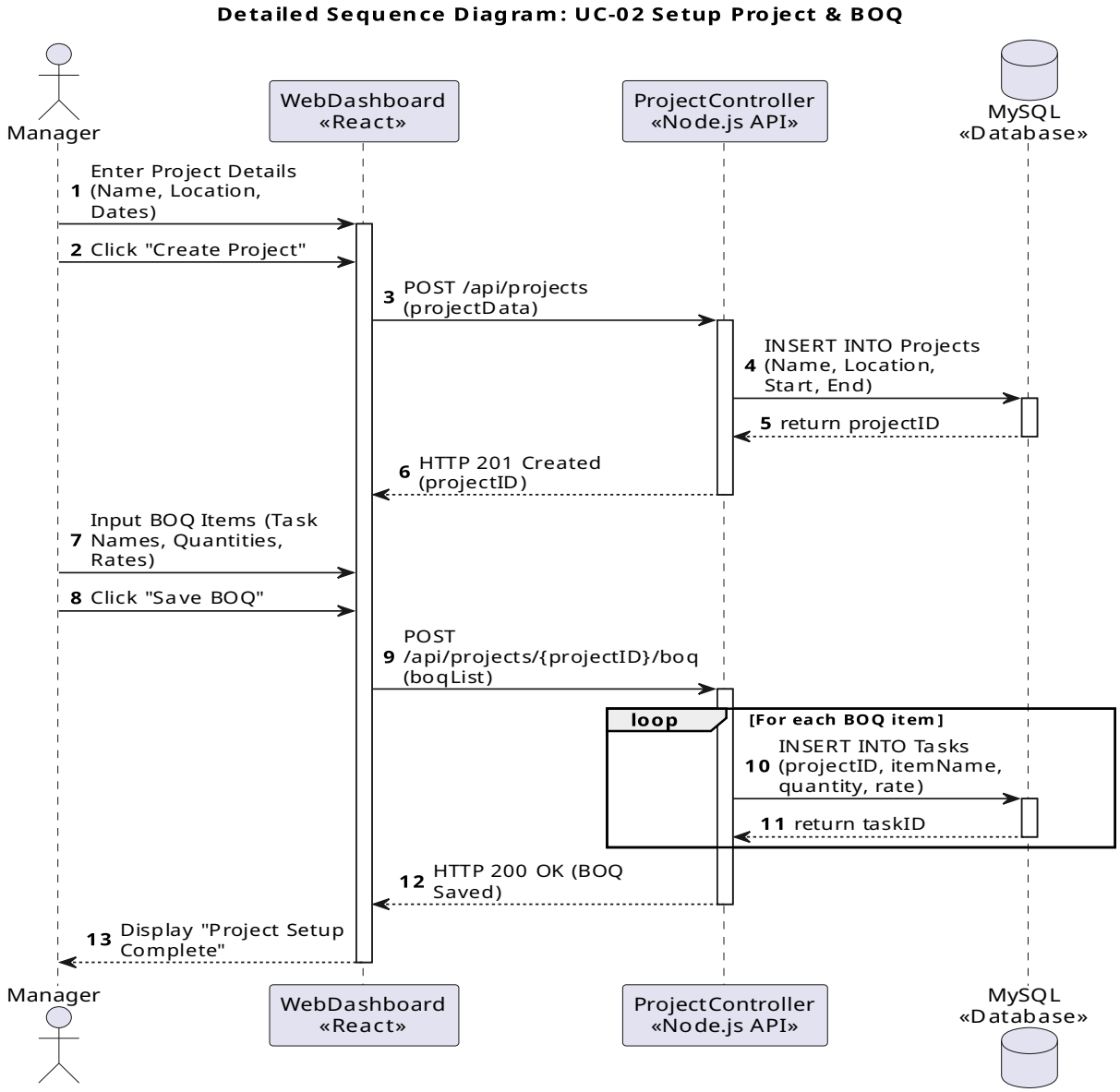
4.3 Sequence Diagrams

The dynamic behaviour of the system is mapped through detailed sequence diagrams. These diagrams model the internal chronological interactions between the frontend UI components (React/Flutter), the Node.js API controllers, the MySQL database, and the S3 storage bucket for all 10 core use cases.

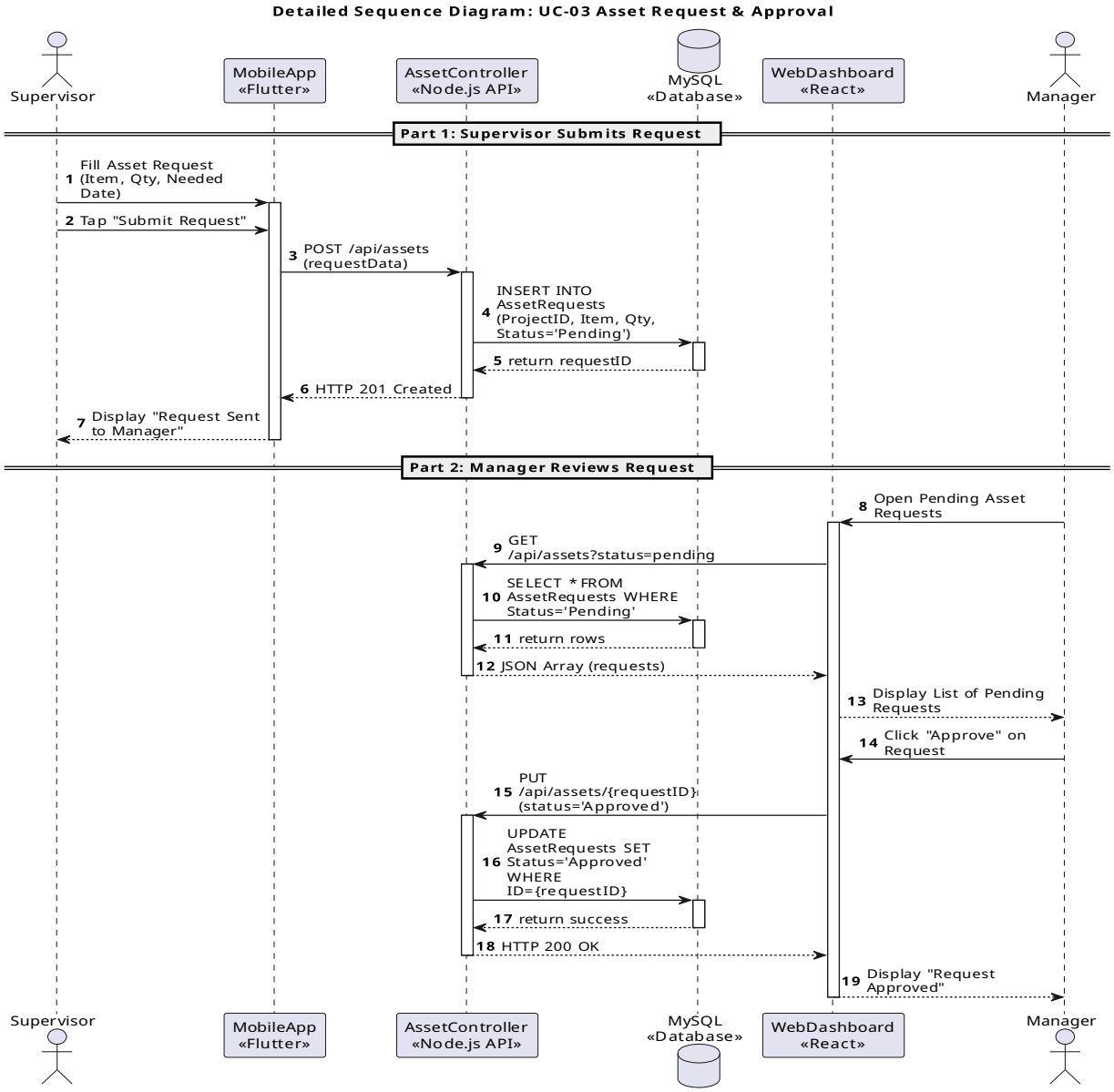
- **UC-01 Submit Daily Report:** Maps the flow of a supervisor selecting tasks, entering labour/materials, and capturing photos, terminating in a concurrent database insertion.



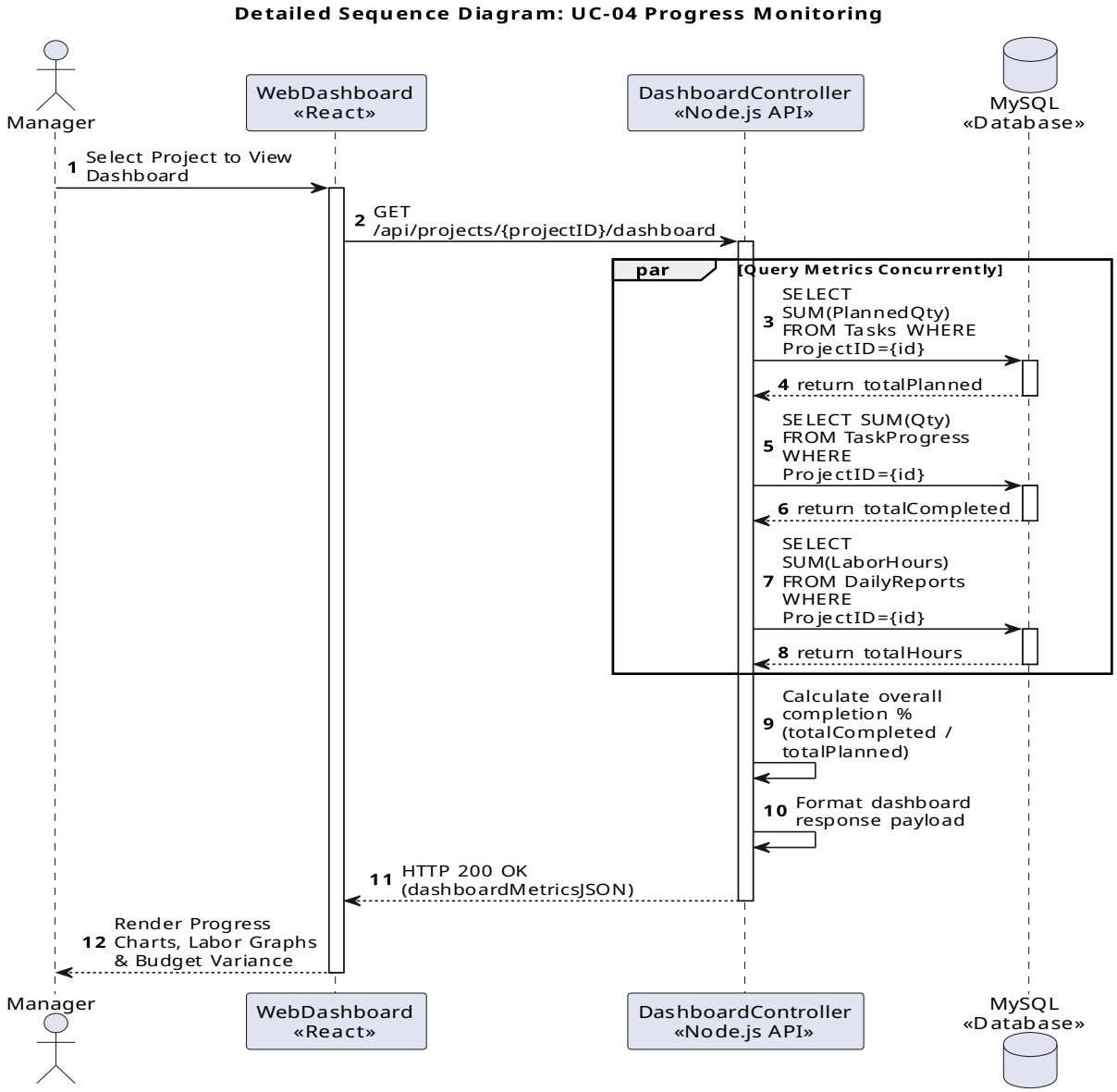
- **UC-02 Setup Project:** Details the manager's process of initializing a project and iterating through a BOQ import to generate individual Task records.



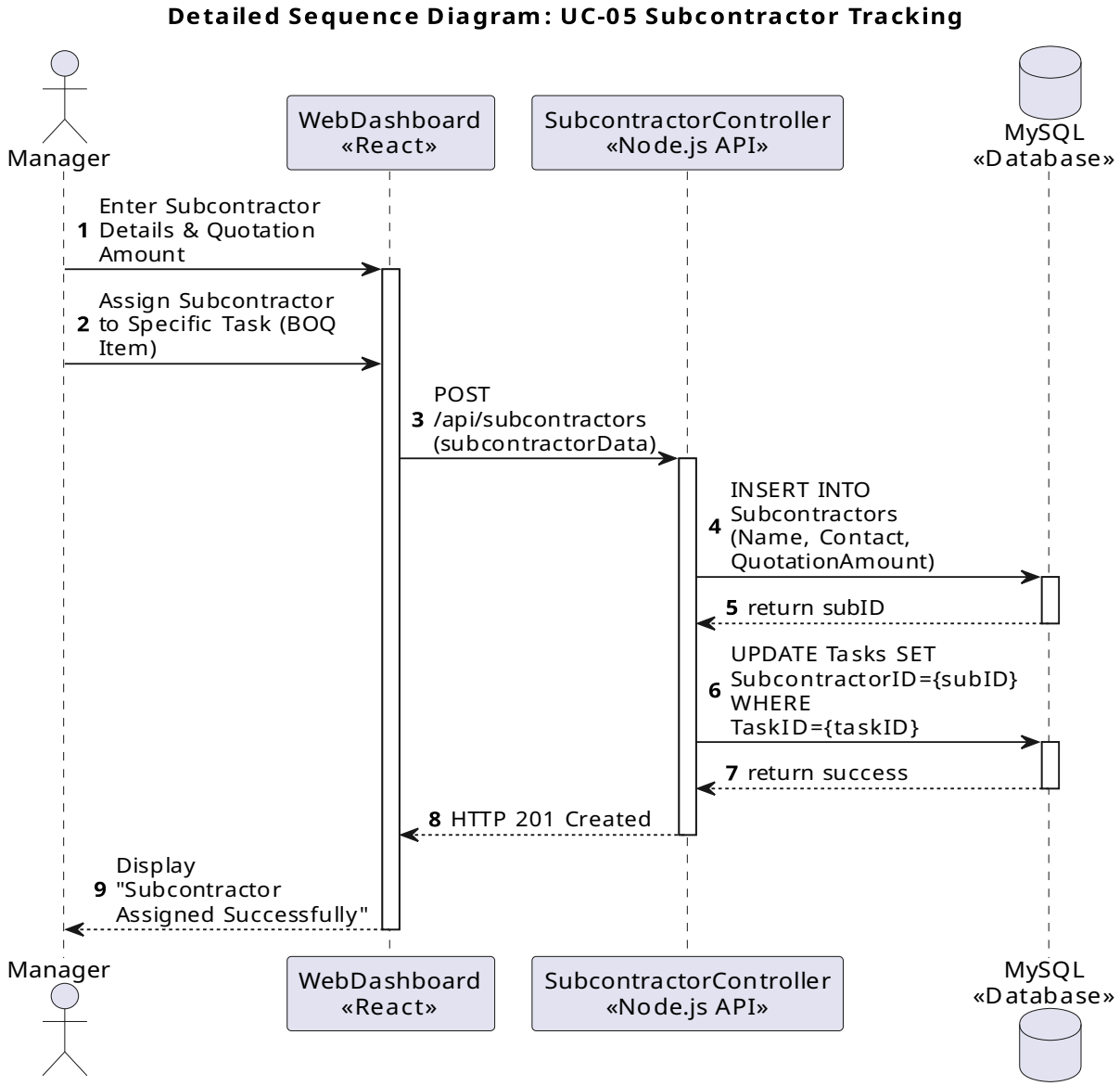
- **UC-03 Asset Request:** Illustrates the two-part asynchronous workflow: the supervisor's initial submission and the manager's subsequent approval or rejection.



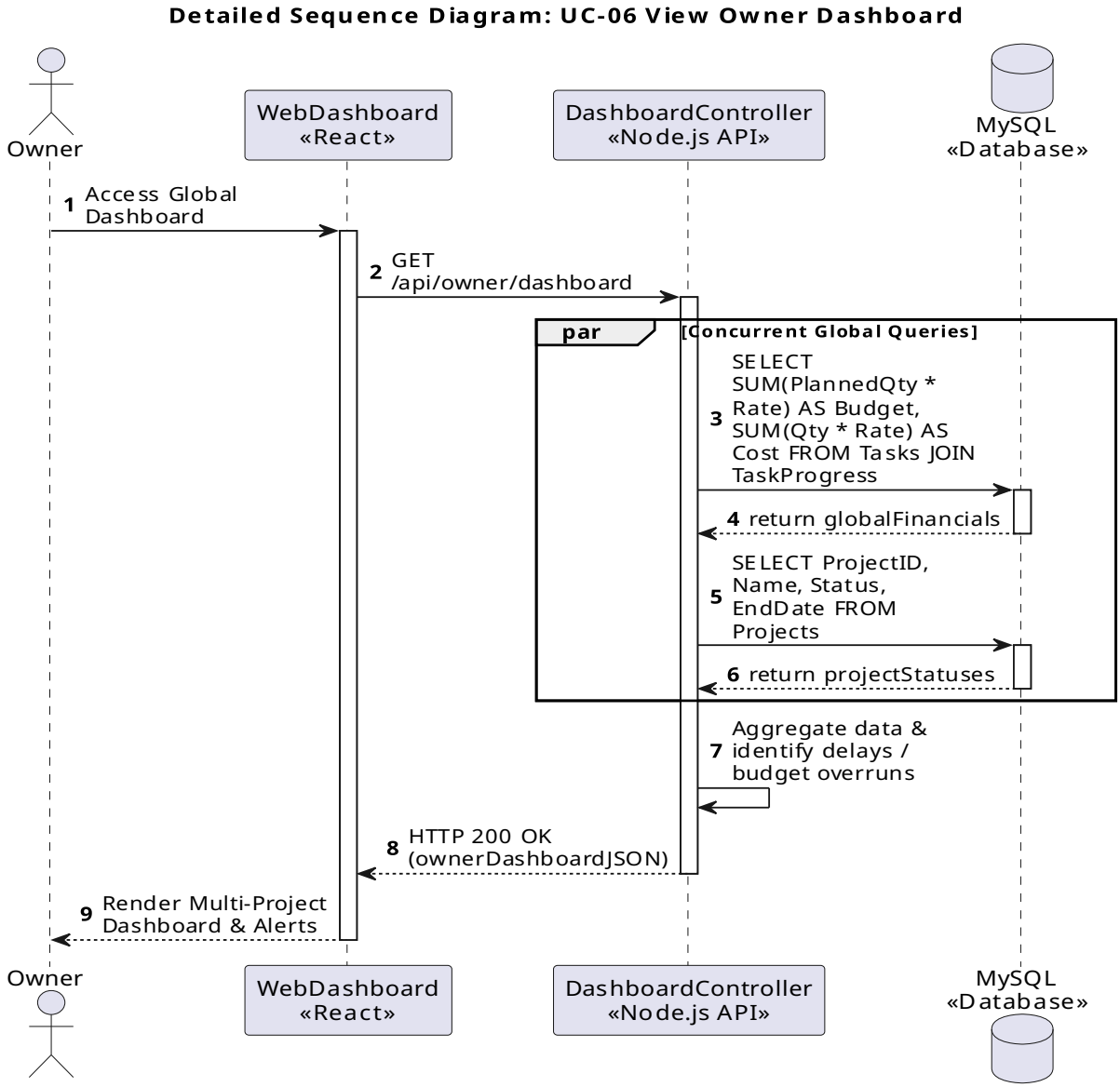
- **UC-04 Progress Monitoring:** Models the manager dashboard querying the API to calculate overall project completion percentages and budget variances dynamically.



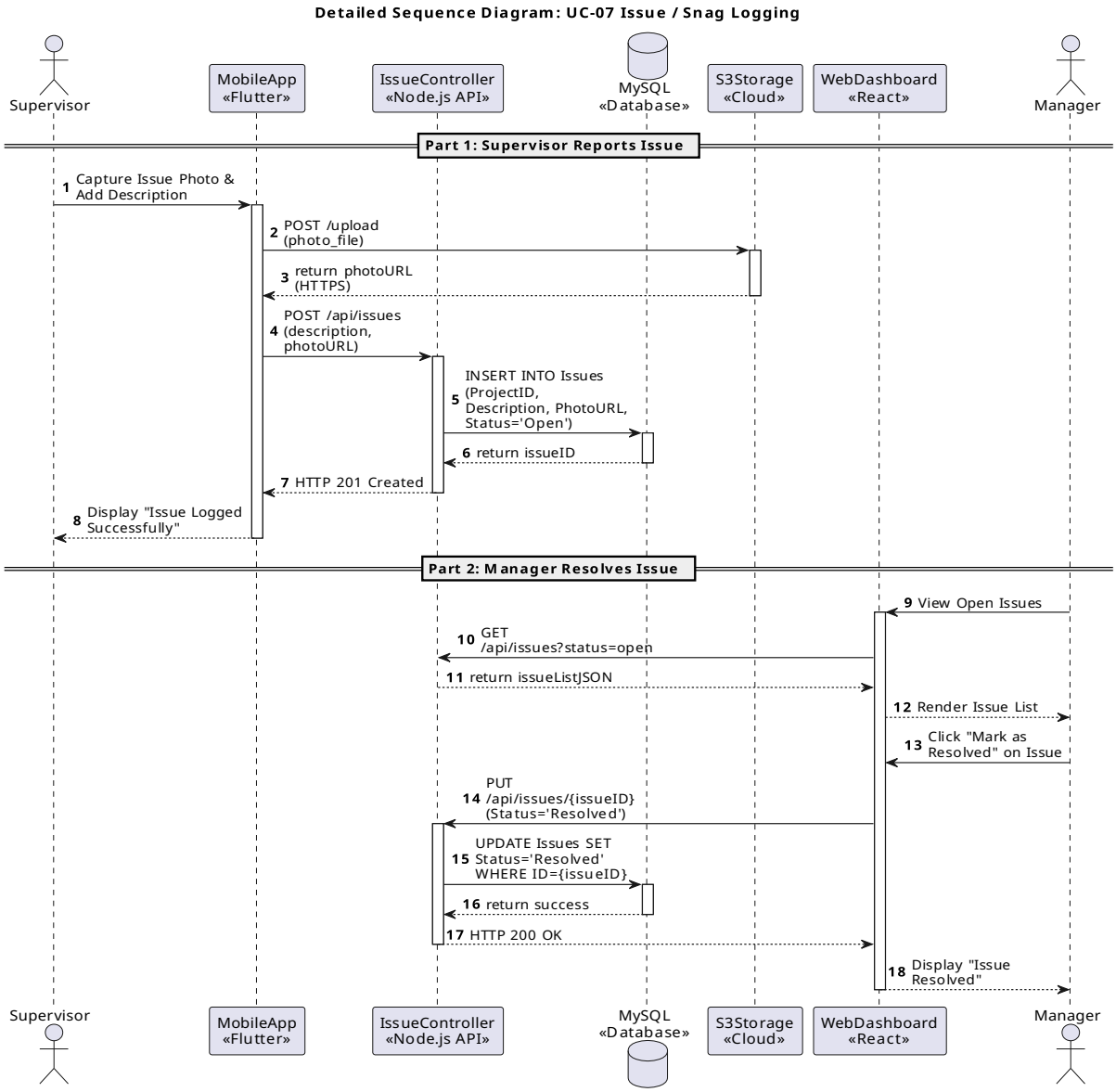
- **UC-05 Subcontractor Tracking:** Shows the assignment of a specific subcontractor to a BOQ task and linking the agreed quotation.



- **UC-06 Owner Dashboard:** Demonstrates the heavy, concurrent global read queries required to aggregate progress and calculate budget alerts across all active projects.

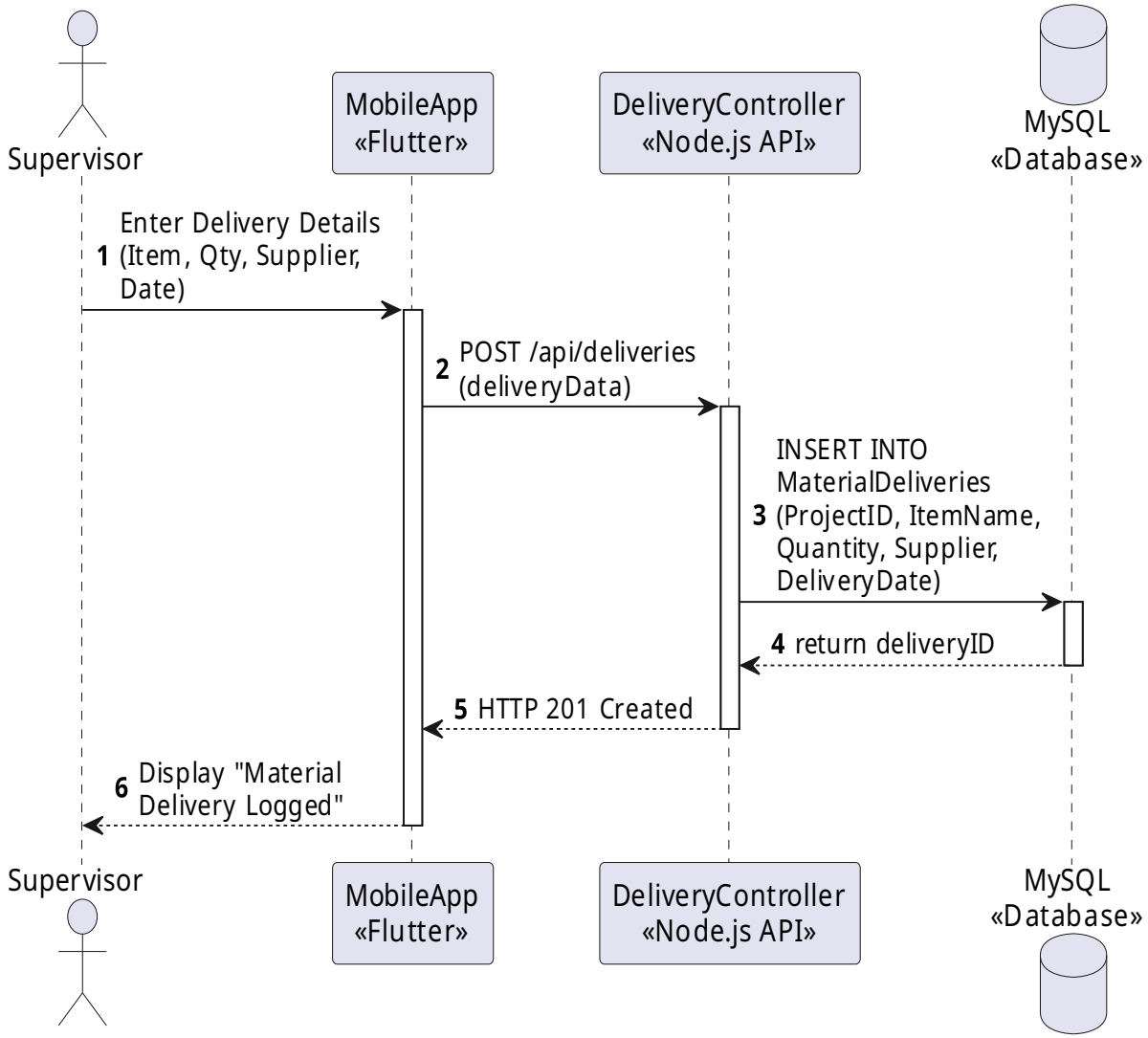


- **UC-07 Issue / Snag Logging:** Maps the flow of a supervisor uploading an issue photo to S3, saving the URL to MySQL, and the manager later resolving the snag.

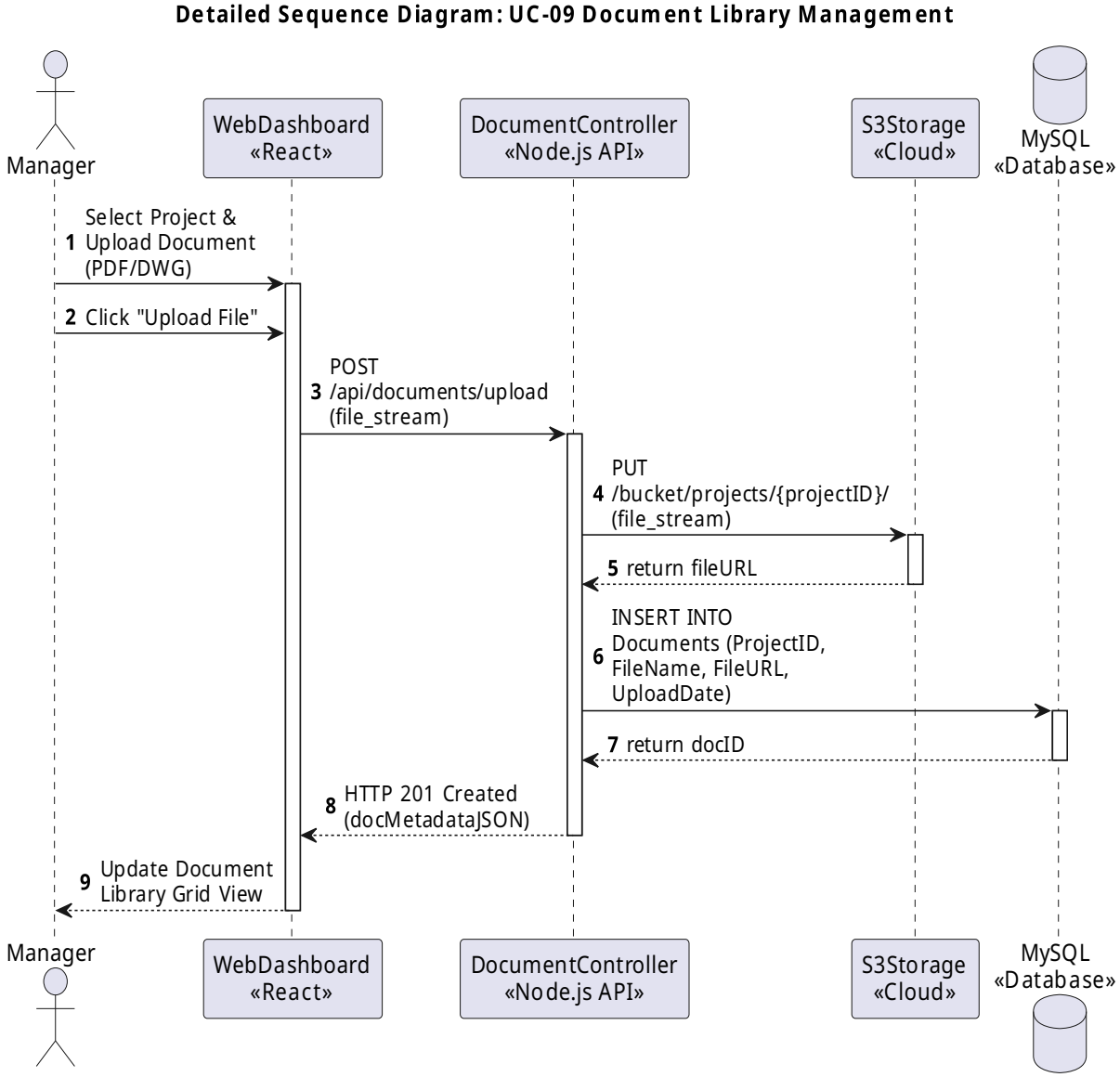


- **UC-08 Material Delivery Log:** Details the independent process of logging arriving materials and supplier information to the database.

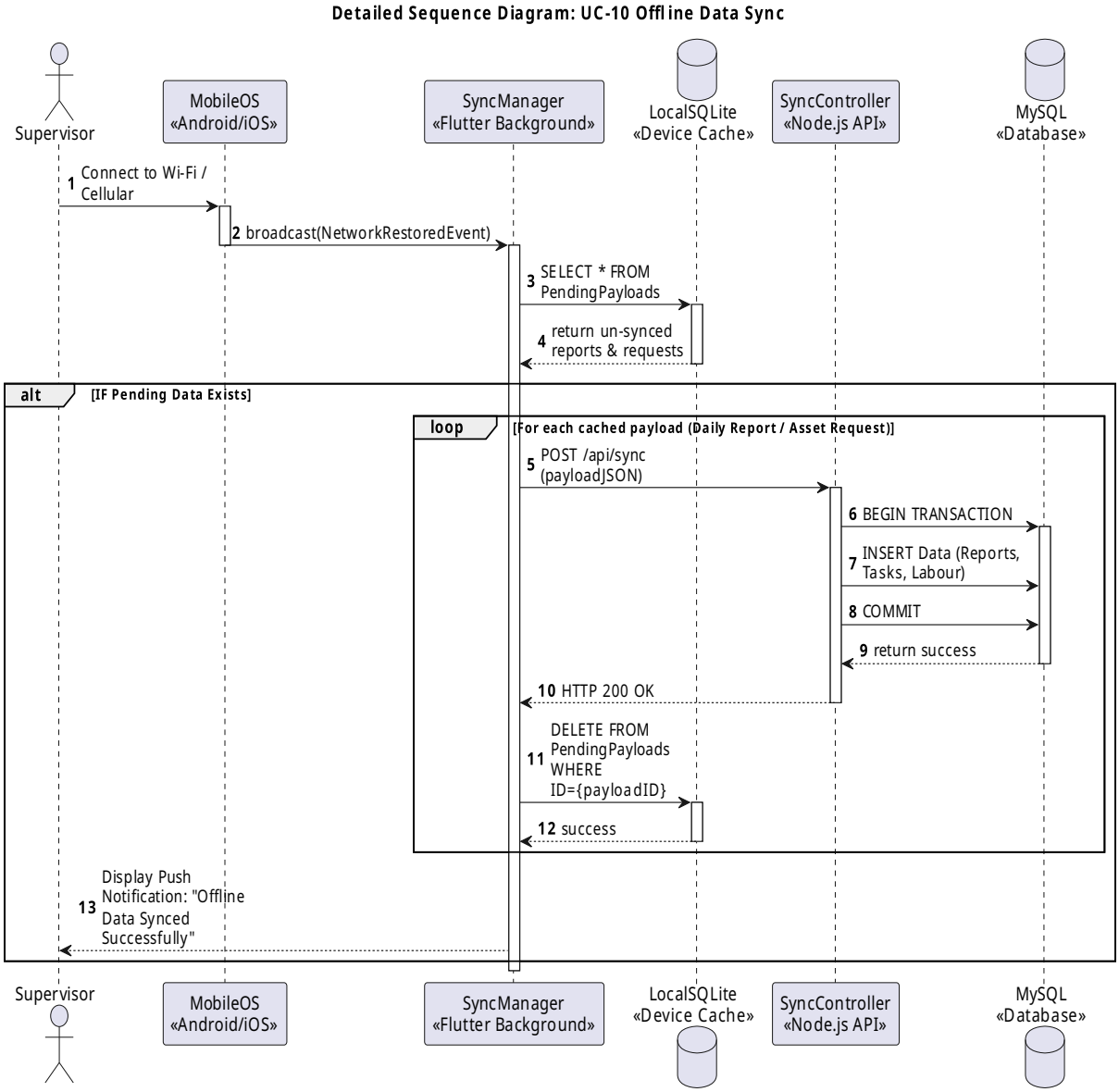
Detailed Sequence Diagram: UC-08 Material Delivery Log



- **UC-09 Document Library:** Illustrates a manager uploading PDFs/DWGs to the S3 bucket and storing the metadata references in the database.



- **UC-10 Offline Data Sync:** Maps the background process where the mobile app detects network restoration and systematically flushes the local SQLite cache to the Node.js API.



4.4 State Machine Diagrams

To manage the lifecycle of mutable data, the system strictly enforces state transitions on two critical entities within the Node.js controllers:

1. **Asset Requests Lifecycle:**
    - Draft (Optional: Saved locally on the mobile device during offline mode).
    - Pending (Persisted to the MySQL database and visible to the Manager).
    - Approved or Rejected (Terminal states triggered by Manager intervention).

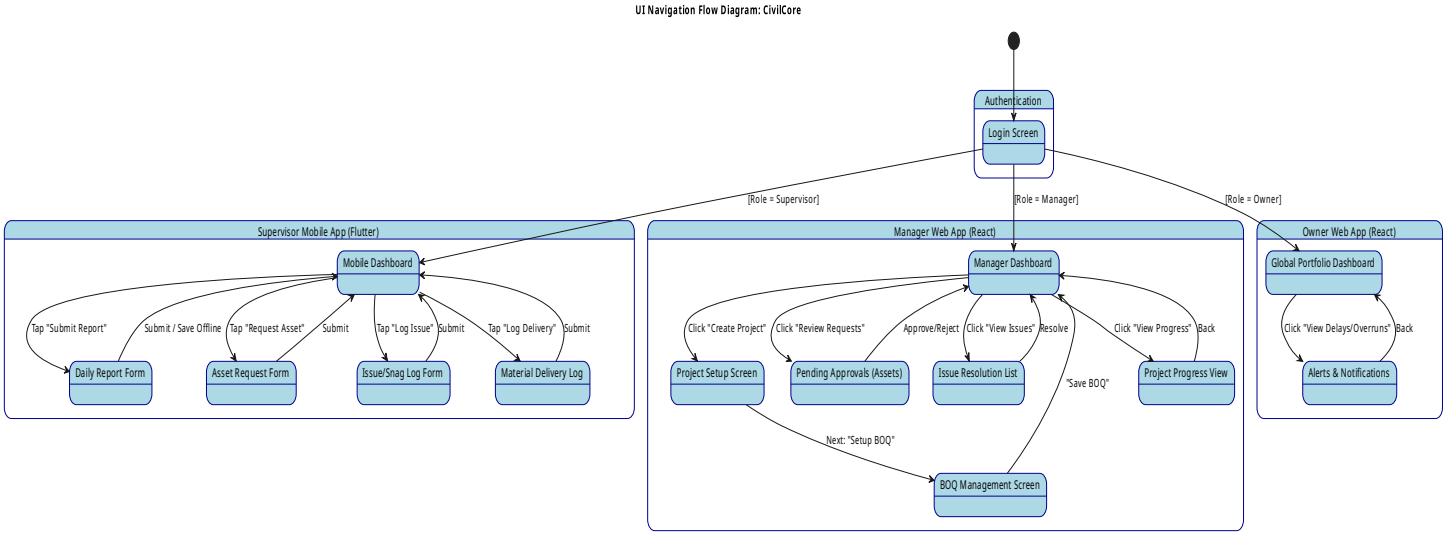
2. **Issue (Snag Log) Lifecycle:**
    - Open (Triggered upon supervisor submission; alerts the manager).
    - Resolved (Terminal state triggered once the manager confirms the physical issue is fixed).

5. UI Design

5.1 Navigation Flow Diagram

The user interface navigation is strictly segregated by the system's Role-Based Access Control (RBAC). Upon authentication, the system automatically routes the user to the appropriate application interface based on their assigned role: Supervisors are directed to the mobile application's home screen, while Managers and Owners are directed to their respective web-based dashboards.

The navigation flow diagram below maps the complete screen-to-screen routing and available user actions within each discrete interface.

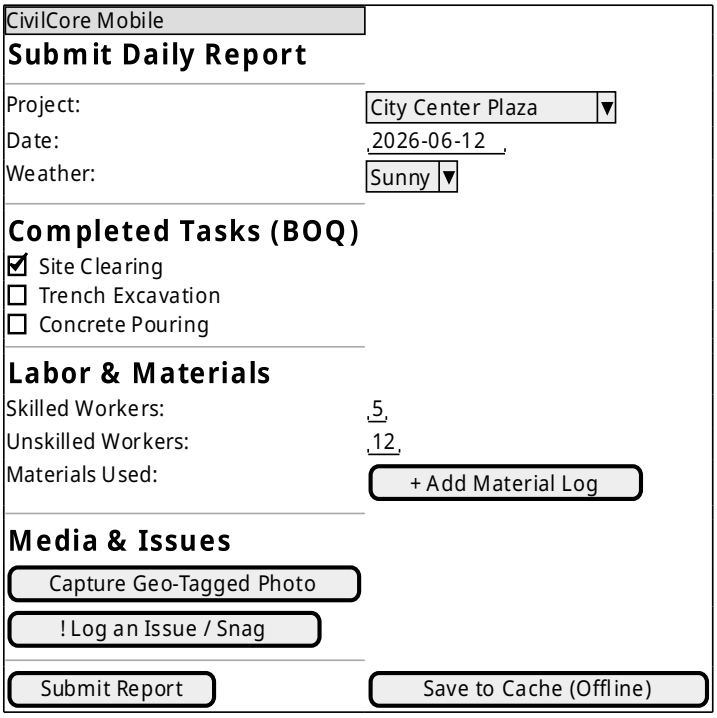


(Note: The source PlantUML file for this diagram is located at diagrams/navigation-flow/navigation-flow.puml)

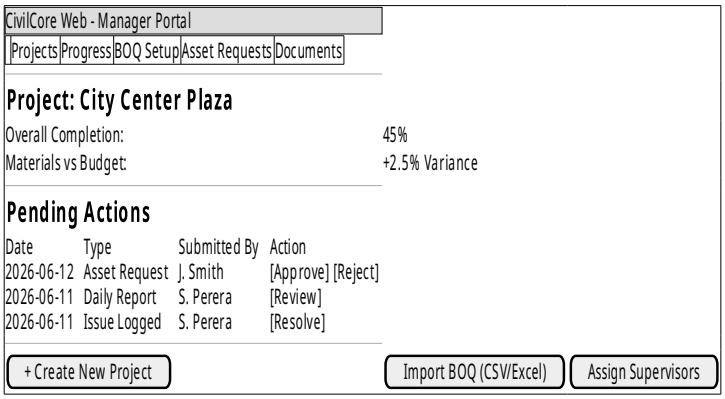
5.2 Wireframes

The system features three primary user interfaces tailored to the workflows of the distinct user roles. The low-fidelity wireframes representing these core screens have been developed and exported to the prototypes repository.

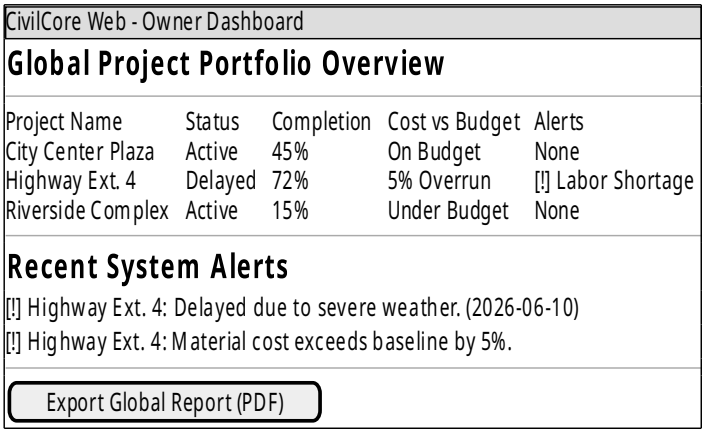
- **Supervisor Mobile Interface (Daily Report View):** Designed for the Flutter mobile app. This interface focuses on capturing the core daily metrics: completed BOQ tasks, labour hours (skilled and unskilled), utilized materials, weather conditions, and geo-tagged site photos. It explicitly includes UI controls for offline caching.



- **Manager Web Interface (Progress & Approvals View):** Designed for the React web application. This interface provides a comprehensive project overview, including a Gantt-style schedule. It highlights overall completion percentages against the BOQ, materials used versus the budget, and provides a consolidated view of pending daily reports, asset requests, and logged issues awaiting review.



- **Owner Web Interface (Global Portfolio View):** A read-only dashboard designed to provide high-level visibility across all active projects. It aggregates cost-versus-budget data and highlights system-generated alerts for critical schedule delays or budget overruns.



5.3 UX Decisions and Rationale

The user experience (UX) strategy for CivilCore was driven entirely by the need to provide a lightweight, mobile-friendly alternative to complex enterprise tools like Primavera P6, which the target users find too cumbersome for daily field use.

- **Mobile App (Supervisors):**
  - **Speed & Efficiency:** To satisfy the strict success criterion that a daily report must be completed in under 5 minutes, the mobile UI heavily utilizes simple checkbox interactions for BOQ tasks rather than manual text entry.
  - **Environmental Considerations:** The interface incorporates high-contrast typography to reduce glare in bright, outdoor site environments. Large touch targets are implemented to accommodate supervisors who may be wearing protective equipment or operating the device while walking the site.
  - **Offline Transparency:** Given the requirement for offline capabilities, the UX explicitly informs the user of their network state. When offline, primary submission buttons dynamically shift to "Save to Cache," preventing user frustration from unexpected network timeouts.
- **Web App (Managers and Owners):**
  - **Data Density & Visualization:** The desktop interfaces leverage the available screen real estate to present dense project data clearly. Complex timelines are visualized using Gantt-style schedules.
  - **Alert Hierarchy:** The Owner dashboard employs strict visual hierarchies (e.g., colour-coded warning banners) to immediately draw attention to critical project anomalies, such as material costs exceeding the baseline budget or weather-related schedule delays.

6. Interface Design

6.1 External Interfaces

The CivilCore system integrates with external cloud infrastructure to manage operations that fall outside the scope of the primary relational database.

- **AWS S3 REST API:** The Node.js backend interfaces directly with AWS S3 (or an equivalent cloud storage provider) via the official AWS SDK to securely handle heavy binary files. This external API is invoked via HTTPS to upload, store, and retrieve geo-tagged site photos captured by supervisors during daily reporting and snag logging. Additionally, this interface supports the project's document library, allowing managers to upload and access large project files such as architectural drawings, subcontractor quotations, and Bill of Quantities (BOQ) files.

6.2 Internal Module Interfaces

The system's Client-Server architecture relies on strictly defined internal boundaries to ensure data integrity and platform decoupling.

- **RESTful Backend API:** The central Node.js backend exposes a stateless, structured REST API that serves as the exclusive communication hub for the system. All data exchange between the Flutter mobile application, the React web dashboards, and the MySQL database occurs through these JSON-over-HTTPS endpoints.
- **Data Exchange Format:** The internal modules communicate strictly using lightweight JSON payloads. This ensures that the complex relational data—such as nested arrays of BOQ tasks, labour hours, and utilized materials—is transmitted efficiently over low-bandwidth mobile networks.
- **Endpoint Routing:** Internal communication is logically organized around core domain entities utilizing standard HTTP methods (GET, POST, PUT). For example, the mobile application transmits end-of-day progress via the /api/v1/reports endpoint, while the web application fetches real-time portfolio metrics for the Owner dashboard via the /api/v1/projects/dashboard endpoint.

7. Security Design

7.1 Authentication and Authorisation Strategy

The CivilCore system employs a stateless authentication mechanism utilizing JSON Web Tokens (JWT). Upon successful login, the Node.js backend issues a cryptographically signed JWT containing the user's unique identifier and their assigned system role (OWNER, MANAGER, or SUPERVISOR). This token is stored securely on the client device and transmitted in the Authorization header of all subsequent API requests.

Authorisation is governed by strict, server-side Role-Based Access Control (RBAC). The backend API utilizes dedicated middleware on every protected route to verify the JWT signature and evaluate the user's role against the endpoint's required permissions. This architecture ensures, for example, that a Supervisor is technically blocked from accessing Manager-specific endpoints—such as approving asset requests or altering a project's Bill of Quantities (BOQ)—even if they attempt to bypass the mobile interface.

7.2 Data Protection

The system ensures data confidentiality and integrity both at rest and in transit:

- **Data in Transit:** All communications between the Flutter mobile application, the React web dashboards, the Node.js API, and the external AWS S3 storage buckets are strictly encrypted using HTTPS/TLS 1.2 or higher. This prevents packet sniffing and man-in-the-middle attacks, which is especially critical when supervisors transmit data over unsecured or public networks near construction sites.
- **Data at Rest:** Sensitive user credentials, specifically passwords, are never stored in plain text. They are irreversibly hashed and salted using the robust bcrypt algorithm before being persisted in the MySQL database. Furthermore, the AWS S3 buckets utilized for storing sensitive project documents (like subcontractor quotations) and geo-tagged site photos are secured using restrictive IAM (Identity and Access Management) policies to prevent unauthorised public access or direct URL enumeration.

7.3 Input Validation Strategy

To maintain data integrity and prevent malformed data from corrupting the MySQL database, the Node.js API implements rigorous server-side input validation.

Given that the system relies on JSON payloads exchanged between decoupled clients and the server, middleware libraries (such as Joi or express-validator) are utilized to define strict validation schemas for every incoming API request. Any request originating from the mobile or web clients that contains missing fields, incorrect data types (e.g., submitting a string instead of a numerical float for BOQ

quantities), or out-of-bound values is immediately intercepted and rejected with an HTTP 400 Bad Request response before the data ever reaches the core controller logic.

7.4 OWASP Top 10 Considerations

The system architecture systematically mitigates critical security risks identified in the OWASP Top 10 framework:

- **Injection Flaws:** Completely mitigated by utilizing parameterized queries and a trusted Object-Relational Mapper (ORM) or Query Builder within the Node.js backend. This approach abstracts raw SQL commands, neutralizing the risk of SQL injection attacks against the MySQL database.
- **Broken Access Control:** Directly addressed by the stateless JWT and RBAC middleware described in Section 7.1. By enforcing access rules on the server rather than relying on the client UI to hide elements, the system prevents privilege escalation.
- **Security Misconfiguration:** Prevented by isolating all sensitive configuration data (such as database credentials, JWT secret keys, and AWS access keys) into environment variables (.env). This ensures secrets are never hardcoded into the source code repository, protecting the system even if the codebase is compromised.

8. Non-Functional Design Decisions

8.1 Reliability and Offline Capabilities

The system architecture explicitly addresses the requirement for supervisors to operate and fill out reports without an active internet connection. The Flutter mobile application integrates local storage to securely cache daily report data, including completed BOQ tasks, labour hours, and geo-tagged photos. A background synchronization manager monitors the device's network state, automatically pushing the cached data to the backend API once connectivity is restored.

8.2 Performance

Performance is optimized across both the data entry and data visualization workflows:

- **Field Reporting:** To satisfy the strict success criterion that a supervisor can submit a complete daily report in under 5 minutes on their mobile device, the application logic and UI are designed to minimize text entry, utilizing rapid selections for BOQ tasks and materials.
- **Dashboard Rendering:** The Owner dashboard must aggregate progress and cost-versus-budget data across all active projects. The Node.js backend processes these heavy read operations using concurrent queries, while the React web application efficiently renders the resulting metrics and alerts without interface lag.

8.3 Scalability

The system is designed to handle concentrated spikes in traffic, particularly when multiple supervisors submit heavy daily reports (containing arrays of tasks, materials, and geo-tagged photos) at the end of the working day. The backend REST API leverages the non-blocking I/O model of Node.js to efficiently process these simultaneous payload submissions without performance degradation.

8.4 Usability

As a lightweight alternative to complex enterprise tools like Primavera P6, the system prioritizes ease of use. The mobile application is tailored for the physical constraints of a construction site, focusing on a streamlined reporting flow that requires a minimal learning curve for field teams.

8.5 Maintainability

By utilizing Flutter for the mobile application, the system achieves true cross-platform capability for both iOS and Android devices from a single unified codebase. This architectural decision significantly reduces the ongoing maintenance burden and ensures feature parity across all supervisor devices.